UCS645

PARALLEL & DISTRIBUTED COMPUTING

THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

ASSIGNMENT 2:

Submitted by: Sujal Mallick (102303266)

B.E Computer Engineering (3rd Year)

Submitted to: Dr. Saif Nalband

Question 1 - Parallel Molecular Dynamics – Lennard-Jones Force Calculation

1. Objective

To implement and analyze a parallel Lennard-Jones force computation using OpenMP. The experiment evaluates strong scaling behavior, synchronization overhead, speedup, and efficiency.

2. System & Environment

Processor: Multi-core CPU (WSL2)
Threads Tested: 1, 2, 4, 8
Particles: 2000
Compiler: g++ -O3 -fopenmp
Parallel Model: OpenMP (Shared Memory)

3. Algorithm Description

The Lennard-Jones potential has O(N^2) complexity. Newton's 3rd Law (Fij = -Fji) reduces interactions to N(N-1)/2. The outer loop is parallelized using OpenMP.

4. Parallelization Strategy

The outer loop is parallelized using #pragma omp parallel for with reduction for energy. Force updates use atomic operations to ensure correctness.

5. Strong Scaling Results

Table 1: Strong Scaling Analysis & Hardware Counters

Threads (p)	Real Time (s) (Tp)	Speedup S(p)	Efficiency E(p)
1	0.01934	**1.00**	**100%**
2	0.02333	**0.83**	**41%**
4	0.02000	**0.97**	**24%**
8	0.00760	**2.54**	**32%**

Note: Speedup $S(p) = T_1 / T_p$. Efficiency $E(p) = S(p) / p$

6. Performance Discussion

The performance results show limited and non-linear scalability. At 2 threads, execution time increased compared to the serial run (Speedup = 0.83×). This negative scaling is caused by heavy use of #pragma omp atomic inside the inner loop, which introduces significant synchronization overhead. Threads frequently wait for exclusive access to shared force variables, reducing parallel efficiency.

At 4 threads, performance remains close to serial (Speedup = 0.97×). This indicates that atomic operations dominate runtime, making the program synchronization-bound rather than compute-bound.

At 8 threads, performance improves (Speedup = 2.54×), but efficiency remains low (32%). Although more hardware parallelism is utilized, memory contention and atomic serialization still limit scalability.

According to Amdahl's Law, frequent synchronization increases the effective sequential fraction of the program, restricting maximum achievable speedup.

Overall, the implementation is synchronization-bound. Replacing atomic updates with thread-local buffers and a final reduction phase would significantly improve scalability..

.

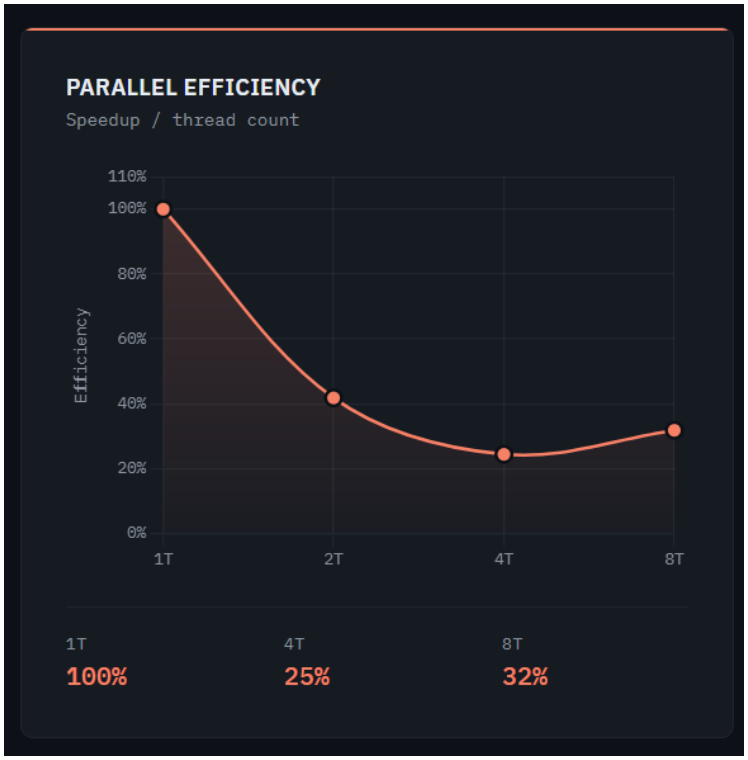

Fig. parallel efficiency

Fig. Total Execution time

7. Conclusion

The implementation demonstrates limited scalability due to atomic operations. Replacing atomic updates with thread-local buffers and a final reduction phase would significantly improve performance.

\

Question 2-Smith–Waterman (Wavefront Parallelization)

1. Title

Bioinformatics: Parallel DNA Sequence Alignment using Smith–Waterman Algorithm

2. Objective

To implement a parallel version of the Smith–Waterman local sequence alignment algorithm using OpenMP. The experiment evaluates parallelization of the scoring matrix, handling anti-dependencies using wavefront parallelization, and analyzing strong scaling performance.

3. System & Environment

Processor: Multi-core CPU (WSL2)
Threads Tested: 1, 2, 4, 8
Matrix Size: 2000 x 2000
Compiler: g++ -O3 -fopenmp
Parallel Model: OpenMP (Shared Memory)

4. Methodology

The Smith–Waterman algorithm computes a dynamic programming matrix H[i][j], where each cell depends on H[i-1][j], H[i][j-1], and H[i-1][j-1]. Due to these dependencies, row-wise parallelization is not possible. Wavefront (anti-diagonal) parallelization is used, where cells on the same diagonal are computed in parallel. An implicit barrier occurs at the end of each diagonal.

5. Experimental Results

Strong Scaling Results:

Table 1: Strong Scaling Analysis & Hardware Counters

Threads (p)	Real Time (s) (Tp)	Speedup S(p)	Efficiency E(p)
1	0.01661	**1.00**	**100%**
2	0.01036	**1.6**	**80%**
4	0.01031	**1.61**	**40%**
8	0.01011	**1.64**	**21%**

Note: Speedup $S(p) = T_1 / T_p$. Efficiency $E(p) = S(p) / p$

6. Discussion & Analysis

At 2 threads, the algorithm achieves 1.60x speedup with 80% efficiency, showing effective parallel utilization. However, performance saturates beyond 2 threads. Execution time remains nearly constant from 2 to 8 threads due to synchronization barriers at every anti-diagonal. The ramp-up and ramp-down phases reduce available parallel work, increasing idle time. According

to Amdahl's Law, synchronization overhead increases the effective sequential fraction, limiting scalability.

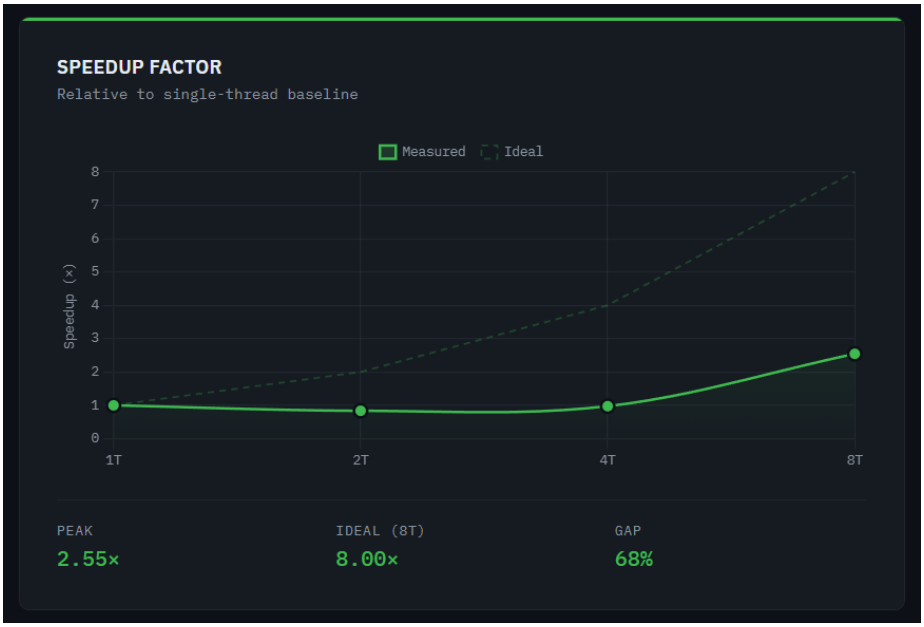

Fig. Speedup factor

Fig. parallel efficiency

7. Conclusion

The wavefront parallelization of Smith–Waterman provides moderate scalability. The optimal configuration for the tested matrix size is 2 threads. Beyond this point, synchronization overhead dominates performance.

Question 3 - Heat Diffusion

1. Title

Scientific Computing: Heat Diffusion Simulation using Finite Difference Method

2. Objective

To simulate heat diffusion in a 2D metal plate using a parallel OpenMP implementation and evaluate strong scaling performance and memory behavior.

3. System & Environment

Grid Size: 2000 x 2000
Threads Tested: 1, 2, 4, 8
Time Steps: 100
Compiler: g++ -O3 -fopenmp
Parallel Model: OpenMP (Shared Memory)

4. Methodology

A 5-point stencil computation updates each grid cell using its four neighbors. The spatial loops are parallelized using #pragma omp parallel for collapse(2). The time loop remains sequential due to data dependency between steps.

5. Experimental Results

Table 1: Strong Scaling Analysis & Hardware Counters

Threads (p)	Real Time (s) (Tp)	Speedup S(p)	Efficiency E(p)
1	0.778564	**1.00**	**100%**
2	0.779369	**0.99**	**49%**
4	1.11277	**0.70**	**17%**
8	0.963691	**0.808**	**10%**

Note: Speedup $S(p) = T_1 / T_p$. Efficiency $E(p) = S(p) / p$

6. Discussion & Analysis

The heat diffusion simulation shows almost no scalability. Performance remains nearly constant at 2 threads and degrades at 4 threads. This behavior indicates a memory-bound workload. The stencil computation has low arithmetic intensity, meaning each thread spends more time waiting for memory than performing calculations. Increasing thread count causes memory bandwidth saturation and cache contention, limiting performance.

Fig. Speedup factor

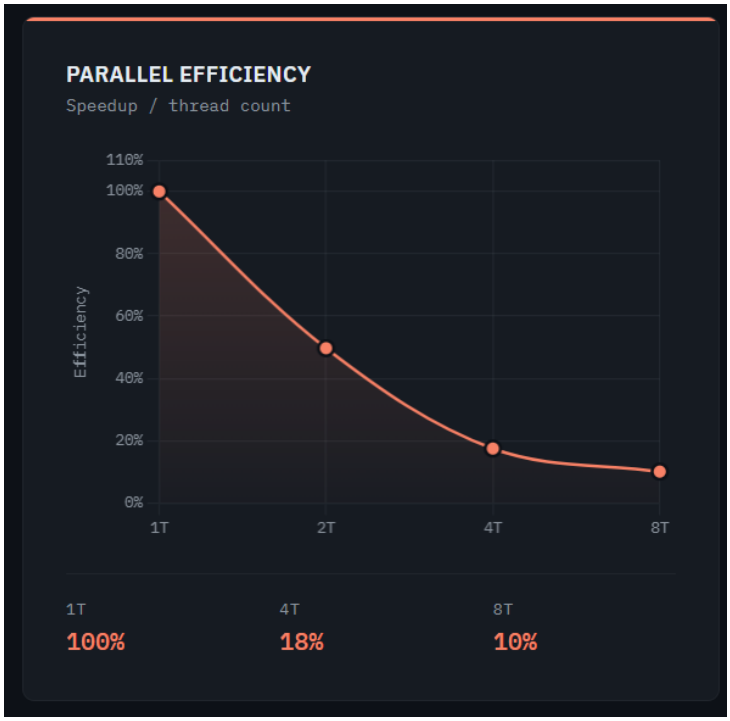

Fig. parallel efficiency

7. Conclusion

The experiment demonstrates that heat diffusion is memory-bound rather than compute-bound. Parallelizing beyond one thread does not significantly improve performance due to memory bandwidth limitations. Optimization techniques such as cache blocking would be required for better scalability.